STAT:5400 Tech Guide 2 — Parallel Computing

Boxiang Wang

2026-10-28

This is a very brief introduction to parallel computing, which makes your code faster by running several processes at the same time. This page also shows how to profile R code using the package `profvis`.

- A computer has several **cores**. Each core can run one process, for example one R session. A laptop usually has 4 to 16 cores. The lnx machines have many more. By default, R uses one core.
- With k cores, the code is at most k times faster. In practice it is less, because starting the processes and sending them data take time.
- We work on one machine, your own computer or one lnx machine.
- Parallel computing is not on the midterm.

To see the number of cores, in a terminal:

```
nproc                 # Linux
sysctl -n hw.ncpu     # Mac
```

In R:

```
parallel::detectCores()
```

1 Introduction to the `parallel` package

This part is based on https://homepage.stat.uiowa.edu/~luke/classes/295-hpc/notes/snow.pdf. The `snow` package (Simple Network of Workstations) by Luke Tierney is now part of the `parallel` package, which comes with R.

- The `parallel` package uses an ordinary R session as the **master** and creates several **workers**.

- Jobs are sent to the workers and the results are returned to the master.

To work on an lnx machine, we first log in and start R:

```
ssh hawkid@l-lnx200.divms.uiowa.edu
R
```

We now create 5 workers on the same machine:

```
library(parallel)
cl <- makeCluster(5)
do.call("rbind", clusterCall(cl, function(cl) Sys.info()["nodename"]))
```

We can stop the workers by `stopCluster(cl)` before we leave R.

1.1 Cluster-level functions

Below are some cluster-level functions.

- Call a function on all workers:

```
clusterCall(cl, exp, 1)
```

- Evaluate an expression on all workers:

```
clusterEvalQ(cl, library(boot))
```

- Apply a function on all workers:

```
clusterApply(cl, 1:3, get("+"), 3)
```

- Send global variables to the workers:

```
x <- 2; y <- 1
clusterExport(cl, c("x", "y"))
clusterApply(cl, 1:3, function(a) a^x + a * y)
```

Note that a worker is a fresh R session. It does not have your variables, your functions, or the packages you loaded. We send them with `clusterExport` and `clusterEvalQ`.

1.2 parLapply

`parLapply` is a parallel version of the `lapply` function. It splits the list approximately evenly across the workers.

```
x <- 1:1000 / 1001
system.time(qtukey(x, 2, df = 2))
system.time(unlist(parLapply(cl, x, qtukey, 2, df = 2)))
```

On a laptop with 16 cores, the first line takes about 6.7 seconds and the second about 1.7 seconds with 5 workers.

1.3 Apply-type functions

There are some apply-type functions we can use:

```
x <- 2; y <- 1
clusterExport(cl, c("x", "y"))
parSapply(cl, 1:3, function(a) a^x + a * y)
parApply(cl, matrix(1:10, ncol = 2), 2, sum)
```

Also `parCapply` and `parRapply`, which apply a function to the columns and the rows of a matrix:

```
(A <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2))
parCapply(cl, A, sum)
parRapply(cl, A, sum)
```

Two notes:

- `set.seed` before `parSapply` does not make the result reproducible, because each worker draws its own random numbers. Use `clusterSetRNGStream(cl, 5400)` instead. It gives each worker its own stream of random numbers.
- On Mac and Linux, `mclapply(x, fun, mc.cores = 5)` does the same job as `parLapply` without creating a cluster. It does not work on Windows.

1.4 A real example: bootstrap

We now illustrate a real example for the bootstrap. We adopt an example that we have used before: how to use the bootstrap to make inference about the sample correlation (lecture note S4P1, the bootstrap t interval). It needs a bootstrap inside a bootstrap.

Using the package `profvis`, we can easily see where the most time is spent:

```
library(profvis)
library(boot)
law <- read.csv("https://boxiang-wang.github.io/stat5400/notes/section4/data/law.csv")

p <- profvis({
  t1 <- Sys.time()
  # estimate cor and the variance of the bootstrap replicates
  mean.se <- function(x, i) {
    corfun <- function(x, i) cor(x$LSAT[i], x$GPA[i])
    v <- var(boot(data = x[i, ], statistic = corfun, R = 2000)$t)
    c(corfun(x, i), v)
  }
  boot.obj <- boot(data = law, statistic = mean.se, R = 200)
  boot.ci(boot.obj, type = "all")
  t2 <- Sys.time()
  difftime(t2, t1, unit = "sec")
})
p
htmlwidgets::saveWidget(p, "profile.html", selfcontained = FALSE)
```

`p` opens the profile in the viewer of RStudio or Positron. The last line also saves it as a web page.

The whole computation takes about 10 seconds on the laptop. The profile shows that almost all the time is spent in the inner `boot`.

Method 1: we use `clusterEvalQ` to evaluate the `boot` function on each worker. Each worker runs 40 outer replicates, and `c` combines the five `boot` objects into one with 200 replicates.

```
invisible(clusterEvalQ(cl, {
  library(boot)
  law <- read.csv("https://boxiang-wang.github.io/stat5400/notes/section4/data/law.csv")
  # estimate cor and the variance of the bootstrap replicates
  mean.se <- function(x, i) {
    corfun <- function(x, i) cor(x$LSAT[i], x$GPA[i])
```

```
    v <- var(boot(data = x[i, ], statistic = corfun, R = 2000)$t)
    c(corfun(x, i), v)
  }
}))
clusterSetRNGStream(cl, 5400)
t1 <- Sys.time()
res <- clusterEvalQ(cl, boot(data = law, statistic = mean.se, R = 40))
cl_boot <- do.call(c, res)
boot.ci(cl_boot, type = c("norm", "basic", "perc", "stud"))
t2 <- Sys.time()
difftime(t2, t1, unit = "sec")
```

Method 2: we write the bootstrap procedure as a function and call `parLapply`.

```
cl_boot_func <- function(...) {
  library(boot)
  law <- read.csv("https://boxiang-wang.github.io/stat5400/notes/section4/data/law.csv")
  mean.se <- function(x, i) {
    corfun <- function(x, i) cor(x$LSAT[i], x$GPA[i])
    v <- var(boot(data = x[i, ], statistic = corfun, R = 2000)$t)
    c(corfun(x, i), v)
  }
  boot(data = law, statistic = mean.se, R = 40)
}
clusterSetRNGStream(cl, 5400)
t1 <- Sys.time()
cl_boot <- do.call(c, parLapply(cl, seq_along(cl), cl_boot_func))
boot.ci(cl_boot, type = c("norm", "basic", "perc", "stud"))
t2 <- Sys.time()
difftime(t2, t1, unit = "sec")
```

Both methods take about 2 seconds with 5 workers. We are done with the cluster:

```
stopCluster(cl)
```

1.5 `foreach`

An alternative to the `parallel` package is `foreach`, which can be used with several parallel computing packages. It writes a parallel loop like a `for` loop. Install `foreach` and `doParallel` first.

```
library(doParallel)
library(foreach)
cl <- makeCluster(3)
registerDoParallel(cl)
foreach(i = 1:3) %dopar% sqrt(i)
foreach(i = 1:3, .combine = c, .errorhandling = "pass") %dopar% {
  sqrt(i)
}
stopCluster(cl)
```

`%dopar%` runs the loop body on the workers. `.combine = c` combines the results into a vector.

2 Distributed learning

Another straightforward way of running code in parallel is batch processing, as we did at the end of Tech Guide 1.

We write a script `distributed.R` and save it in a directory, say `~/5400/parallel`.

- The script gets the arguments `nodeid`, `seedid`, `R1`, and `R2` from the command line. `R1` and `R2` are the numbers of inner and outer bootstrap replicates.
- The script saves the whole R image in an `.RData` file. This is a good practice, since we can trace back if there is an error, unless the file is too large.

Paste the whole block into a terminal to create the file:

```
mkdir -p ~/5400/parallel
cd ~/5400/parallel
cat > distributed.R <<'EOF'
# load arguments
args <- commandArgs(TRUE)
if (length(args) == 0) stop("No arguments supplied.")
for (a in args) eval(parse(text = a))
if (!exists("nodeid")) stop("no nodeid")
if (!exists("seedid")) stop("no seedid")
if (!exists("R1")) stop("no R1")
if (!exists("R2")) stop("no R2")

set.seed(seedid)
t1 <- Sys.time()
library(boot)
```

```
law <- read.csv("https://boxiang-wang.github.io/stat5400/notes/section4/data/law.csv")
# estimate cor and the variance of the bootstrap replicates
mean.se <- function(x, i) {
  corfun <- function(x, i) cor(x$LSAT[i], x$GPA[i])
  v <- var(boot(data = x[i, ], statistic = corfun, R = R1)$t)
  c(corfun(x, i), v)
}
boot.obj <- boot(data = law, statistic = mean.se, R = R2)
t2 <- Sys.time()
print(difftime(t2, t1, unit = "sec"))
save.image(file = paste0("image_", nodeid, ".RData"))
EOF
```

In the shell, we run the script 10 times, each with its own seed:

```
R1=2000; R2=100
ndmax=10; nodeid=1
while [ $nodeid -le $ndmax ]; do
  nice -n19 Rscript --no-save distributed.R "seedid=$nodeid; nodeid=$nodeid; R1=$R1; R2=$R2"
  nodeid=$((nodeid + 1))
done
wait
ls
```

- We run the R script 10 times on 10 cores. Check the number of cores with `nproc` first. On a shared machine, do not use all of them.
- Each run does `R2 = 100` outer replicates. We combine them later to get 1000 replicates in total.
- `> $nodeid.out` sends the output of each run to its own file, for example `3.out`.

To combine the results, we run the following R code in the same directory:

```
ids <- 1:10
all_boot <- vector("list", length(ids))
# get the boot object from each R image
for (ii in seq_along(ids)) {
  load(paste0("image_", ids[ii], ".RData"))
  all_boot[[ii]] <- boot.obj
}
# combine them together and get the bootstrap CI
library(boot)
all_boot_c <- do.call(c, all_boot)
```

```r
all_boot_c$R
boot.ci(all_boot_c, type = c("norm", "basic", "perc", "stud"))
```

3 Exercises

✋ By hand

1. Run the `qtukey` example with `makeCluster(k)` for $k = 1, 2, 4, 8$ and record the time for each k. Plot the time against k.
2. Run the bootstrap example (Method 2) with 2, 4, and 8 workers. Is the time halved each time the number of workers doubles?
3. Log in to an lnx machine. Print its number of cores. Run the distributed script with `ndmax=5` and watch `top` in a second terminal.

4 References

- R Core Team, *Package parallel*. In R, type `vignette("parallel")`.
- Luke Tierney, *Simple Network of Workstations for R*.
- The help page of `boot`, section *Parallel operation*.